Course Release Tools

Purpose

The course release tools separate authoring from publication. A working course repository can contain chapter source, speaker notes, slide source, code that has not yet been released, and other instructor material. The active course is the student-facing subset. It contains only the chapters and supporting files that have been selected for release.

This distinction matters for two reasons. First, the working repository remains the complete source of truth. Second, publishing a new chapter is an explicit operation. Material does not become public merely because it exists in the working repository.

The tools support this model without tying the build logic to one class. Course names, chapter lists, repository addresses, colors, file exclusions, and legacy URLs live in each course's `release.json`. The shared programs in this repository interpret that configuration and produce the site.

The release model

Each class uses three related locations:

1. The working repository contains all authoring material and all course code.
2. The working repository's `release/` directory is a generated preview of the student site.
3. The active repository contains the published subset and is served by GitHub Pages from its root.

The working repository is authoritative. Files in the active repository should be exact copies of their counterparts in `release/`. The intended difference is which chapters and files are present, not different versions of the same file.

The normal flow is:

```
working sources -> generated release -> local review -> active repository -> GitHub Pages
```

The tools currently build and check the first two steps. Copying the reviewed release into the active repository remains a deliberate operation so the result can be inspected before it becomes public.

Repository organization

The shared tools live in `~/courses/tools`:

```
tools/
  build_course_release.py
  check_course_release.py
  migrate_course_layout.py
  new_course.py
  code_viewer.html
  md-to-pdf/
  COURSE_RELEASE_TOOLS.md
```

Each working course repository contains its own `release.json`. It may also keep small `tools/build_release.py` and `tools/check_release.py` launchers so the same commands work from inside every class.

This arrangement puts shared behavior in one repository while leaving course policy with the course. A change to link checking or code display is made once. A change to the Advanced Database chapter list does not affect Structure of Programming Languages.

Starting a new course

The new-course tool creates the standard working and active repositories, an initial `release.json`, shared-tool launchers, authoring directories, and an empty generated course site. For example:

```
~/courses/tools/new_course.py operating-systems \
    --title "Operating Systems" \
    --description "Processes, memory, files, concurrency, and operating-system design."
```

By default, the command creates `~/courses/operating-systems/working` and `~/courses/operating-systems/active`. Both are initialized as local Git repositories with an initial commit. The working repository begins with an unpublished `chapter-00-introduction` scaffold. The active repository contains only the generated student site and its repository metadata.

Each working chapter keeps its Markdown source and generated PDF directly in `chapter-NN-title/`. Its `code/`, `slides/`, and `readings/` directories hold the related material. The initial slide Markdown and speaker notes are placeholders; they are not a built deck. Only finished components explicitly listed in `release.json` enter the student site. Index pages are generated in `release/`.

The scaffolder refuses to replace a nonempty course directory. This makes it safe to rerun a command after correcting its arguments without risking an existing class.

GitHub creation is separate because it changes external state. Add `--github` when that is intended. The default owners create a private working repository under `gregdelozier` and a public active repository under `kentcs`, both using the course slug as the repository name. The command also enables GitHub Pages for the active repository, served from `/` on its `main` branch. Owner names and the base directory are command-line options.

Converting existing courses

`migrate_course_layout.py COURSE` previews the move from separate chapter, code, and slide collections to chapter directories. `--apply` first saves a complete backup outside the working repository, including uncommitted work. Use `--backup PATH` to select a new destination. A JSON report beside the backup records file moves and rewritten references.

The migration updates source paths in `release.json` without adding released chapters or changing public filenames. It also updates local path references in documentation and build scripts. Existing runnable code is preserved byte for byte. Slide source and HTML use `slides-NN-title` filenames; speaker notes remain separate. Configured readings and matching lecture notes move beside their chapter.

Number conflicts are errors, not an invitation to guess. For older code from a different chapter sequence, the explicit `--archive-conflicting-code` option moves it to `archive/legacy-code`. Unknown directories, symlinks, and destination file collisions require review. Running the script again after a completed migration makes no changes. An interrupted migration should be recovered from its backup before retrying.

The migration never touches active repositories or pushes anything. Compare a `--skip-build` release before and after migration, then run full builds and the release checker. The first comparison verifies that changing the organization has not changed which files students receive. Full builds verify that the moved sources and build scripts still work.

Building a course release

From anywhere, run the shared builder with the working repository path:

```
~/courses/tools/build_course_release.py ~/courses/structures/working
```

From a configured course repository, the compatibility launcher is shorter:

```
python3 tools/build_release.py
```

The builder reads `release.json`, runs any requested chapter or slide builds, and assembles the selected files. It creates chapter landing pages and code browsers, copies readings and slide assets, and writes the top-level course index. It builds into a temporary sibling directory first. The existing `release/` directory is replaced only after the new build completes.

Slide decks are also exported to PDF using Reveal.js print layout and the same browser runtime already used for Mermaid. Each PDF has one slide per page and excludes speaker notes. Chapter contents pages link both HTML and PDF versions. Setting `"pdf": false` on a slide entry disables that export; otherwise an existing slide PDF is required when using `--skip-build`.

Root-level `extras/` supplies supplemental rendered documents and their supporting assets. The builder generates its index only when public files exist. Markdown, Mermaid source, instructor notes, and speaker notes stay in working. Files and directories starting with `.private` are excluded at every depth, independent of course configuration. The release checker rejects these private files even if they are inserted into the output manually.

The slide and chapter sources include a small author line, `Gregory S. DeLozier, PhD`, under the title. New-course templates include the same byline. It is part of the source material, so rebuilding does not remove it.

For a quick preview that uses already-built PDFs and slides:

```
python3 tools/build_release.py --skip-build --output /tmp/course-preview
```

This is useful while changing site layout or release configuration. It avoids rebuilding documents whose source has not changed and leaves the normal release tree alone.

Checking a release

Run the checker after every build:

```
python3 tools/check_release.py
```

For a preview in another directory:

```
python3 tools/check_release.py --site /tmp/course-preview
```

The checker verifies that local links resolve, expected chapter directories are present, private source formats have not entered the site, released slide HTML does not contain speaker notes, and copied code files match their working-course sources byte for byte.

Application templates may contain routes such as `/pets` or template expressions that are meaningful only when the application runs. Those files are code, not site navigation, so the checker does not interpret links inside template directories as release-site links.

Configuring a course

The `release.json` file has two jobs. It describes the site, and it identifies the material to release.

The `site` object contains the course introduction, repository links, code-card description, and theme colors. Course-specific details remain here rather than inside the shared Python program.

The `chapters` array is the release boundary. Each entry may provide:

- a chapter PDF and an optional command that builds it;
- a slide deck, its build command, and supporting assets;
- a code directory;
- readings, including Markdown readings that must first become PDFs.

Removing a chapter from this list removes it from the generated release. Adding one makes it eligible for publication. This is why the configuration should be reviewed as carefully as the generated site.

The `release` object defines names and suffixes omitted while copying code. It is used to exclude authoring files, caches, local environments, database journals, and similar material. `legacy_urls` can preserve previously published workbook, slide, or topic paths when a course changes organization.

Markdown to PDF

The `md-to-pdf` directory contains the document publishing pipeline used by the course builders. The command is available at:

```
~/.local/bin/md-to-pdf chapter.md
```

By default it writes `chapter.pdf` beside the Markdown source. An explicit output and an additional print stylesheet are also supported:

```
md-to-pdf chapter.md -o preview/chapter.pdf
md-to-pdf chapter.md --css course-print.css
```

Pandoc parses the Markdown into a document tree. The builder resolves local images relative to the source file and identifies Mermaid code blocks. Mermaid CLI renders those diagrams as Scalable Vector Graphics (SVG), preserving clean lines and text in the PDF. Pandoc then produces HTML, and WeasyPrint applies the shared print stylesheet and lays out Letter-sized pages.

The pipeline stages the completed PDF and replaces an existing output only after a successful render. A failed build therefore leaves the previous PDF intact. Missing local images and rendering errors stop the build instead of silently producing an incomplete document.

The source, lock files, templates, and styles are tracked in Git. Installed Node modules and the Python virtual environment live in the same directory for local use but are excluded from the repository. The paths under `~/.local` are compatibility symlinks, so existing chapter build scripts do not need to change.

Publishing safely

A normal release should follow this sequence:

1. Build the working course’s release.
2. Run the release checker.
3. Open the generated `index.html` locally and inspect the affected chapters.
4. Check PDFs and served slide decks, not only their source files.
5. Copy the reviewed subset into the active repository.
6. Confirm that active files match the generated release.
7. Commit and push the active repository.

The build and validation steps are intentionally separate. A successful build means the requested files could be assembled. A successful check means the assembled result satisfies the release rules that can be tested automatically. Neither replaces visual review of documents, slides, and pages.